

Cybersecurity

Back-End Component Vulnerabilities

Lesson 14.2



Class Objectives

By the end of today's lesson, you will be able to:

- 1 Differentiate between front-end and back-end component vulnerabilities.
- 2 View confidential files with a directory traversal attack by using the dot-dot-slash method.
- 3 Exploit a web application's file upload functionality to conduct a local file inclusion attack.
- 4 Modify a web application's URL to use a malicious remote script to conduct three different remote file inclusion attacks.



Intro to Back-End Component Vulnerabilities



Let's Recap

Web vulnerabilities are weaknesses that exist within the intended purposes of a web application.

Malicious actors can exploit these web vulnerabilities to cause unintended consequences.



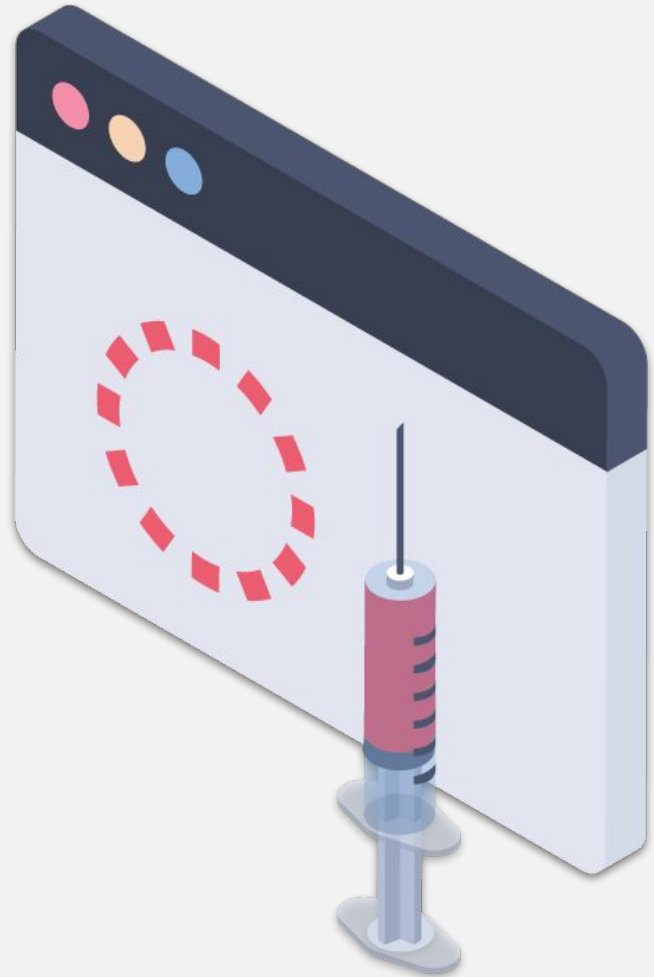
The OWASP Top 10: Injection

A01	Broken Access Control	A06	Vulnerable and Outdated Components
A02	Cryptographic Failures	A07	Identification and Authentication Failures
A03	Injection	A08	Software and Data Integrity Failures
A04	Insecure Design	A09	Security Logging and Monitoring Failures
A05	Security Misconfiguration	A10	Server-Side Request Forgery (SSRF)

Recap: Injection

Injection is a specific type of attack that occurs when an attacker supplies untrusted input to an application.

The **payload** is processed as part of a query or command that alters the intended functionality of a program.

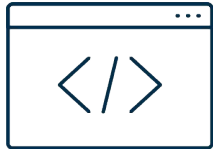


Review: Injection Types

There are various types of injection depending on the architecture and functionality of the application.

Cross-Site Scripting (XSS)

- Conducted by an attacker inserting malicious JavaScript into a web application.
- Depends on the application modifying the client-side code with a user's input.



SQL Injection

- Conducted by an attacker inserting malicious SQL statements into a web application.
- Depends on the application running queries against a SQL database.

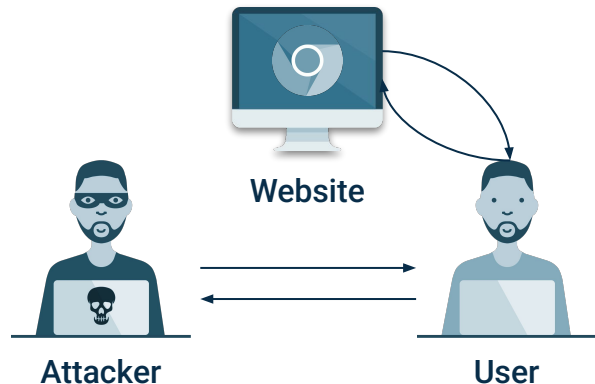


Review: Cross-Site Scripting (XSS)

The types of XSS include:

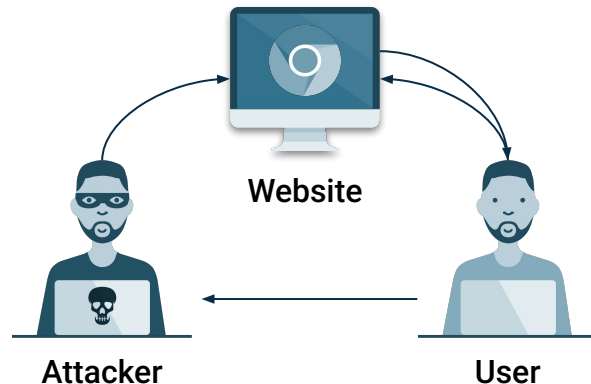
Reflected (Non-Persistent) XSS

Depends on the user input being immediately returned to the user and not stored on the application's server.



Stored (Persistent) XSS

Depends on the user input being stored on the application's server and later retrieved by a user accessing the web application.



Review: Input Validation

Input validation is a method used to validate the data input with a predefined logic to ensure that the input is what the application is expecting.

EXAMPLE:

Email:

 Please include an '@' in the email address. 'user.email.com' is missing an '@'.

Range:

Date: 

Time: 



Back-End Component Vulnerabilities

Last Lesson

In the last lesson, we covered **JavaScript** and **HTML** cross-site scripting vulnerabilities.

These languages are considered **front-end components**, as they form the part of the web application that the user interacts with, typically with a browser.



Another front-end component is **Cascading Style Sheets (CSS)**, which formats the layout of the HTML.



Today's Lesson

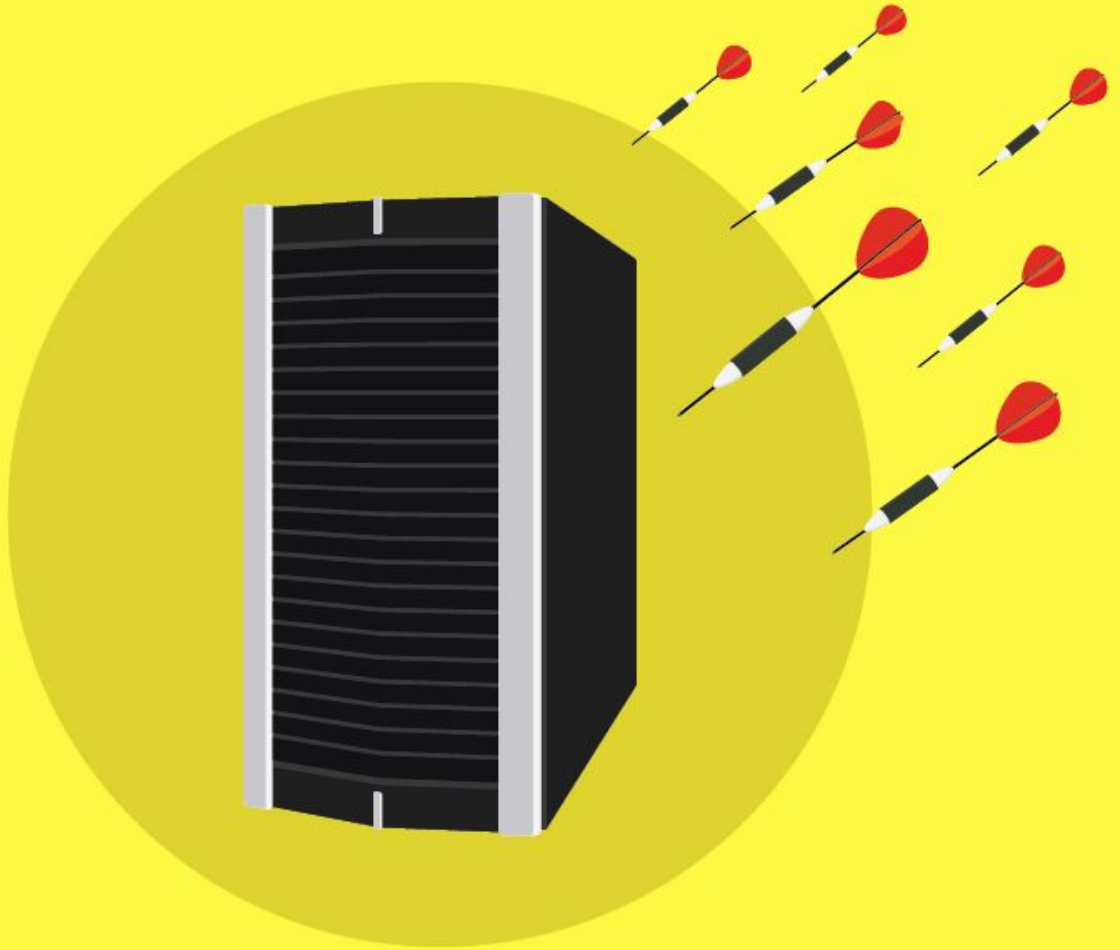
We will focus on vulnerabilities that can exist within back-end components. These components apply the business logic of how the application works, including:

- 1 File structure
- 2 Content management
- 3 Access control

Back-end languages include **PHP** and **Java**.

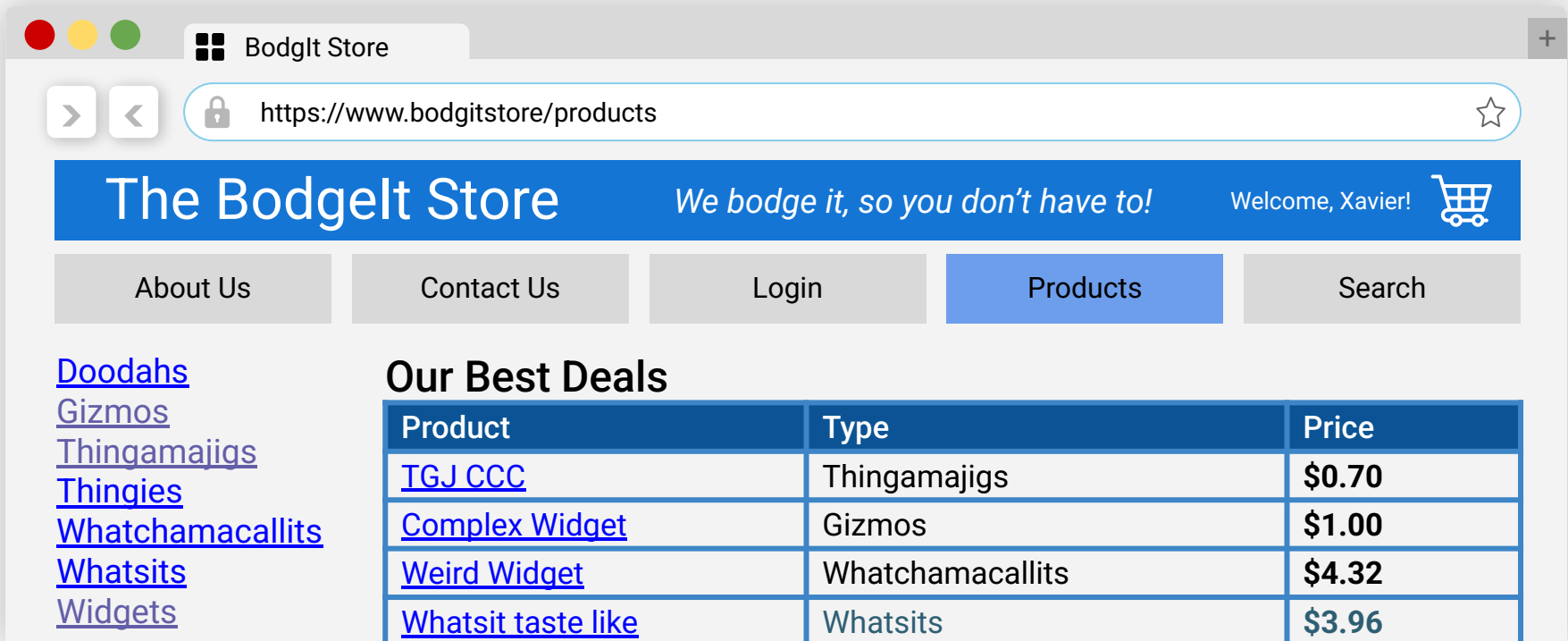


Vulnerabilities in **back-end components** could potentially allow a malicious actor to access or display confidential files or documents that exist on an organization's private server.



Vulnerabilities: Intended Purpose

An **intended purpose** of an application involves a user accessing a webpage and displaying prices of several products at an online store. The query that retrieves the product prices is the applied business logic.

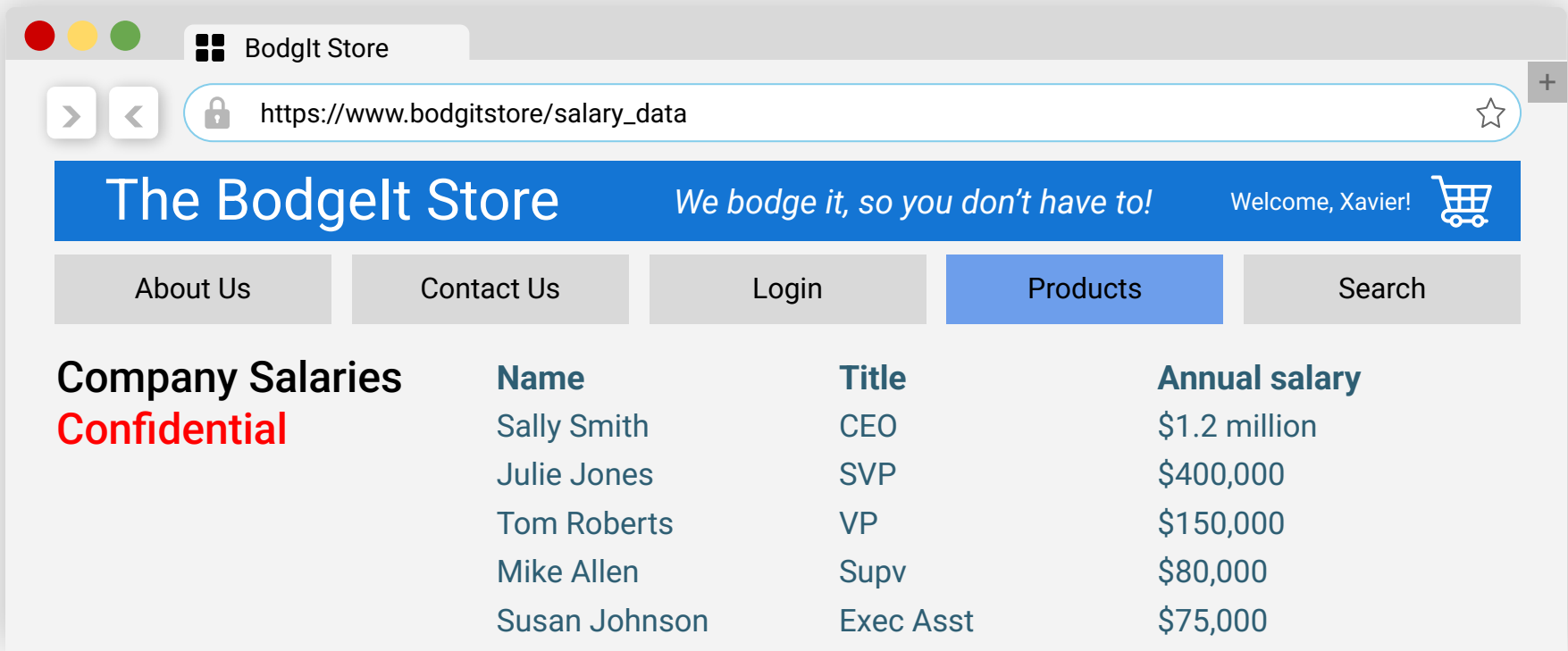


The screenshot shows a web browser window with the address bar displaying <https://www.bodgitstore/products>. The website header features the logo "The Bodgelt Store" with the tagline "We bodge it, so you don't have to!" and a welcome message "Welcome, Xavier!". A navigation bar includes links for "About Us", "Contact Us", "Login", "Products" (which is highlighted), and "Search". On the left side, there is a vertical list of product links: "Doodahs", "Gizmos", "Thingamajigs", "Thingies", "Whatchamacallits", "Whatsits", and "Widgets". The main content area is titled "Our Best Deals" and contains a table with three columns: "Product", "Type", and "Price".

Product	Type	Price
TGJ CCC	Thingamajigs	\$0.70
Complex Widget	Gizmos	\$1.00
Weird Widget	Whatchamacallits	\$4.32
Whatsit taste like	Whatsits	\$3.96

Vulnerabilities: Unintended Consequence

A malicious actor has changed the URL to access confidential files, such as employee salaries, which are **NOT intended** to be displayed to users.



The screenshot shows a web browser window with a single tab titled "Bodgelt Store". The address bar displays the URL `https://www.bodgitstore/salary_data`, which is not the intended public URL. The website header features the logo "The Bodgelt Store", the tagline "We bodge it, so you don't have to!", a user greeting "Welcome, Xavier!", and a shopping cart icon. A navigation bar contains links for "About Us", "Contact Us", "Login", "Products" (which is highlighted), and "Search". The main content area displays a table titled "Company Salaries" with a red "Confidential" warning. The table lists five employees: Sally Smith (CEO, \$1.2 million), Julie Jones (SVP, \$400,000), Tom Roberts (VP, \$150,000), Mike Allen (Supv, \$80,000), and Susan Johnson (Exec Asst, \$75,000).

Company Salaries	Name	Title	Annual salary
Confidential	Sally Smith	CEO	\$1.2 million
	Julie Jones	SVP	\$400,000
	Tom Roberts	VP	\$150,000
	Mike Allen	Supv	\$80,000
	Susan Johnson	Exec Asst	\$75,000

Today's Lesson

We will cover three attacks that exist within back-end component vulnerabilities:

First

We will begin by demonstrating how an attacker can use a **directory traversal attack** to access an organization's confidential data.

Next

We will learn how an attacker can use a common web application file upload functionality to conduct a **local file inclusion attack**.

Finally

We will demonstrate another file inclusion attack, called a **remote file inclusion attack**, in which an attacker can reference remote malicious scripts through the URL.



Important



The techniques we will learn throughout this unit can be used to cause serious damage to an organization's systems.

This is **ILLEGAL** when done without permission. All of the labs that we provide are safe locations to test the methods and tools taught during the week.

NEVER apply any of these methods to any web applications that you do not own or do not have clear, written permission to be interacting with.



Directory **Traversal**



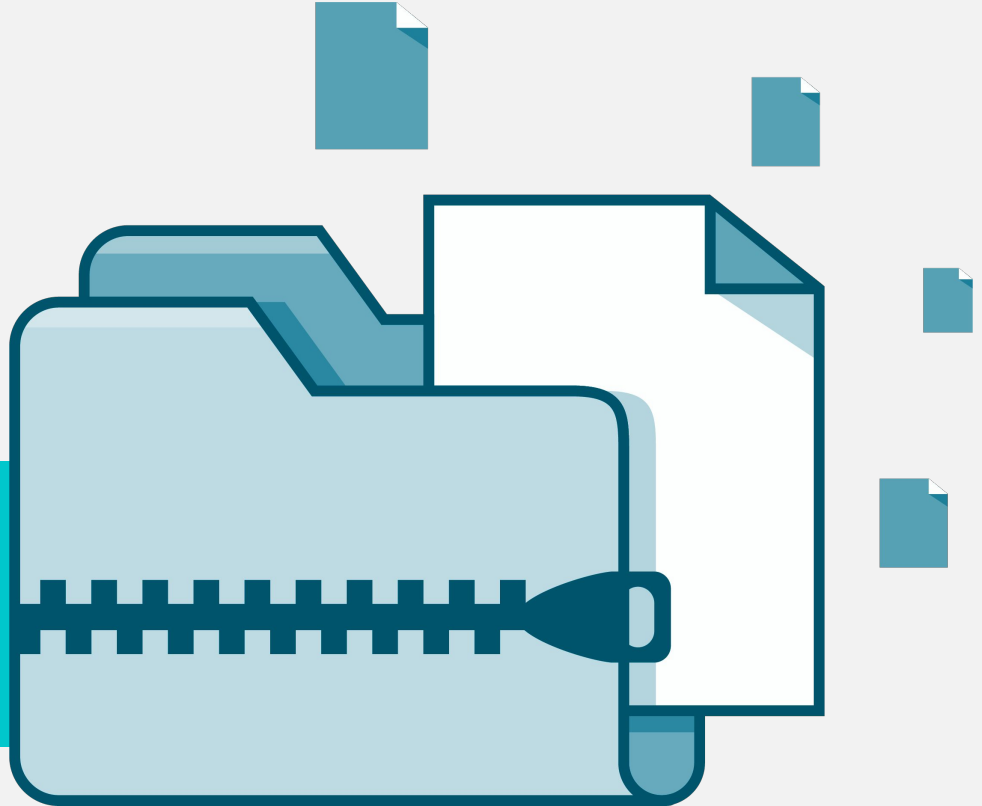
Directory traversal, also known as path traversal, occurs when an attacker accesses files and directories from a web application outside a user's authorized permissions.

Directory Traversal

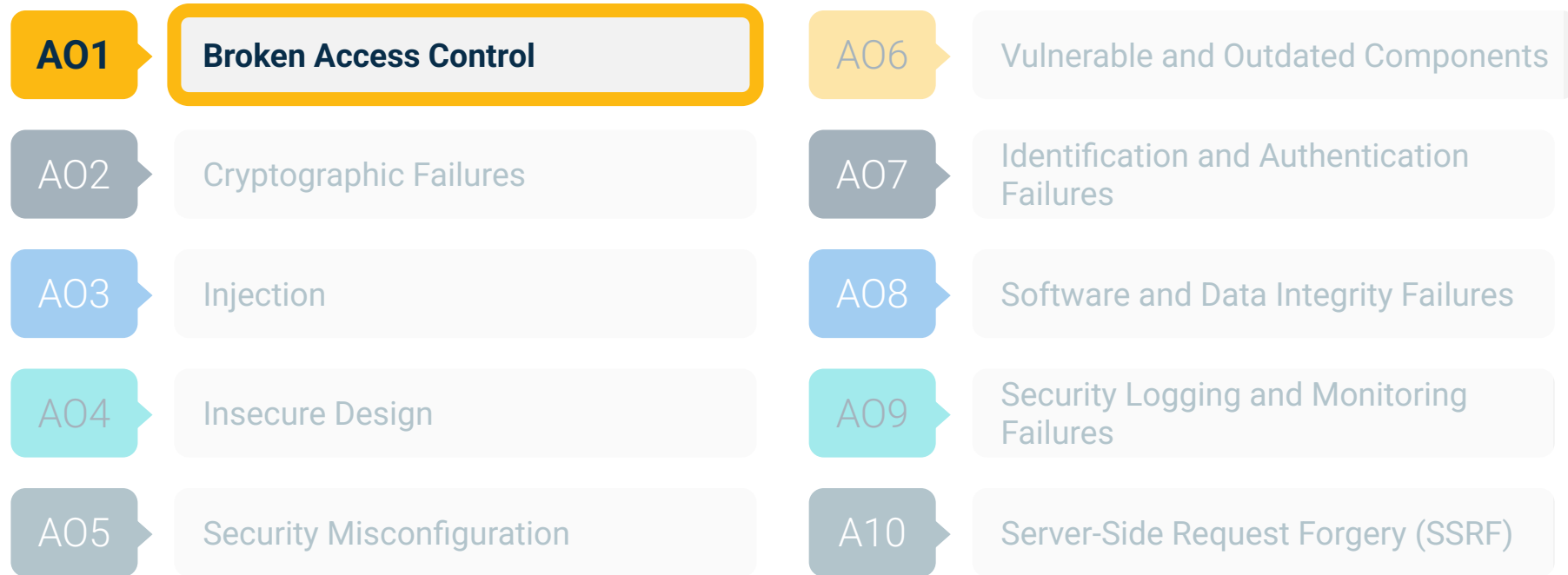
Using this attack, a malicious actor could access confidential business documents, source code, or critical system files that exist within an organization's private server directly from a web application.



Directory traversal primarily falls under the OWASP Top 10 risk category **broken access control**.



The OWASP Top 10: Broken Access Control



Per OWASP, broken access control is explained as follows:

Restrictions on what authenticated users are allowed to do are often not properly enforced.

Attackers can exploit these flaws to access unauthorized functionality and/or data, such as access other users' accounts, view sensitive files, modify other users' data, change access rights, etc.”



Understanding Directory Traversal

We first need to understand what happens behind the scenes as an application interacts with the file structure of a web application.

We'll cover the following:

01

The intended purpose of the web application.

02

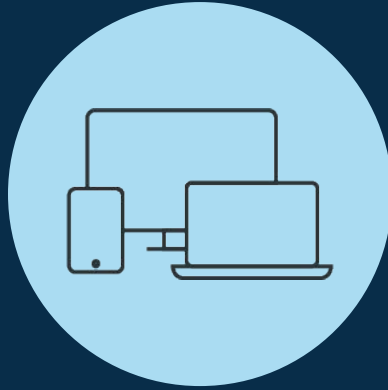
How the application accesses files and directories from a web server.

03

The modification of the URL to access unintended files.

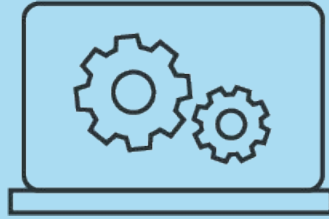
04

The traversal of directories to access additional unintended files with the dot-dot-slash method.



Instructor Demonstration

Directory traversal



Real-World **Challenges,** **Impact,** and **Mitigations**

Access to Filesystem Structure

The demo illustrated how modifying a URL with the dot-dot-slash method can cause unintended results.

In our demonstration...

We were able to preview back-end systems to display the file structures.

But in the real world...

Application security analysts would likely not have access to the filesystems.

For example:

If `../../../../etc/passwd` doesn't display the contents, we can try to keep adding the dot-dot-slashes until the file is displayed:

```
../../../../etc/passwd
```

```
../../../../etc/passwd
```

```
../../../../etc/passwd
```

Variety of Confidential Files and Data

While the demonstration involved one file “/etc/passwd,” many other system files or documents might exist that are not intended to be viewed by an attacker, such as:

`/etc/group`

To view information on groups that exist on the server.

`/etc/hosts`

To view information on how IPs are mapped to hostnames.

Impact

The impact of directory traversal attacks include allowing malicious actors to do the following:



View an organization's confidential data.

View an organization's system files to determine other vulnerabilities.

Help an attacker launch other attacks.

Mitigation Methods

Because a directory traversal attack is conducted by using the dot-dot-slash method to modify user input like the URL, methods to mitigate this attack include:

- 1 Limiting user input when calling for files from the web application.
- 2 Using **input validation** to limit the user's ability to modify the file.
- 3 Running web servers under a special service user account.

Directory Traversal Summary

Intended purpose of original function	A web application interacts with its web server to access intended files.
Vulnerability and method of exploit	With the directory traversal attack, a user can modify the user input using a dot-dot-slash method to access unintended files in other directories.
Unintended consequence of the exploit	Confidential documents or system files can be accessed directly from the web application by a malicious user.
Mitigation of the vulnerability	Mitigation includes applying input validation code logic or limiting user input when calling for system files.
Potential impact of the exploit	The impact could include unauthorized parties accessing an organization's confidential data within its servers. This unauthorized data could be used to launch other attacks.



Questions?





Activity

Directory traversal

In this activity, you'll continue to play the role of an application security analyst at Replicants, inspecting the company's web application for web vulnerabilities, specifically directory traversal.

Suggested Time
20 minutes





Time's Up!
Let's review

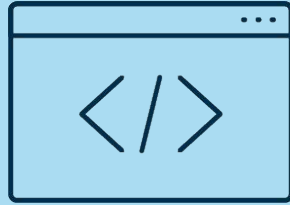


Questions?





Break
15 mins



Web Application **Back-End Code**

Web Application Back-End Code

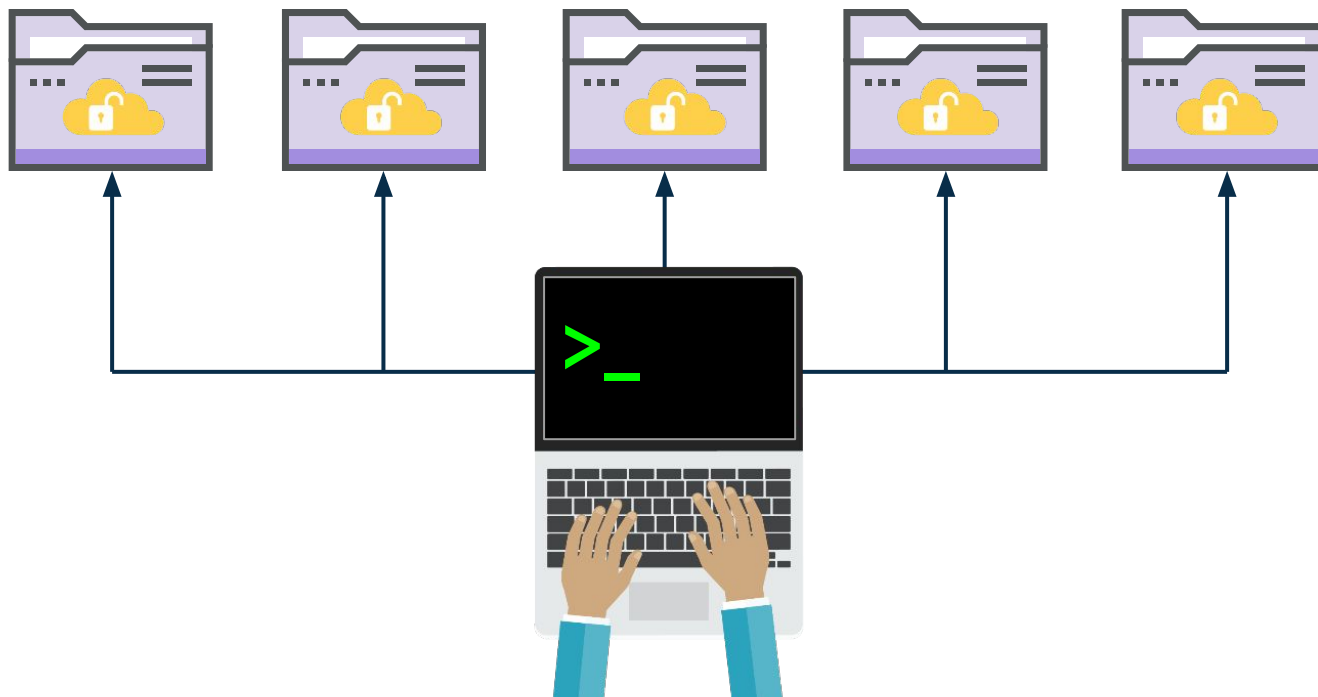
Web applications use **back-end components** to apply business logic to the application. Back-end components can provide file structure, content management, and access control, and include back-end languages such as **PHP** and **Java**.



- Web application vulnerabilities can exist within back-end components.

Directory Traversal Has Limited Impact

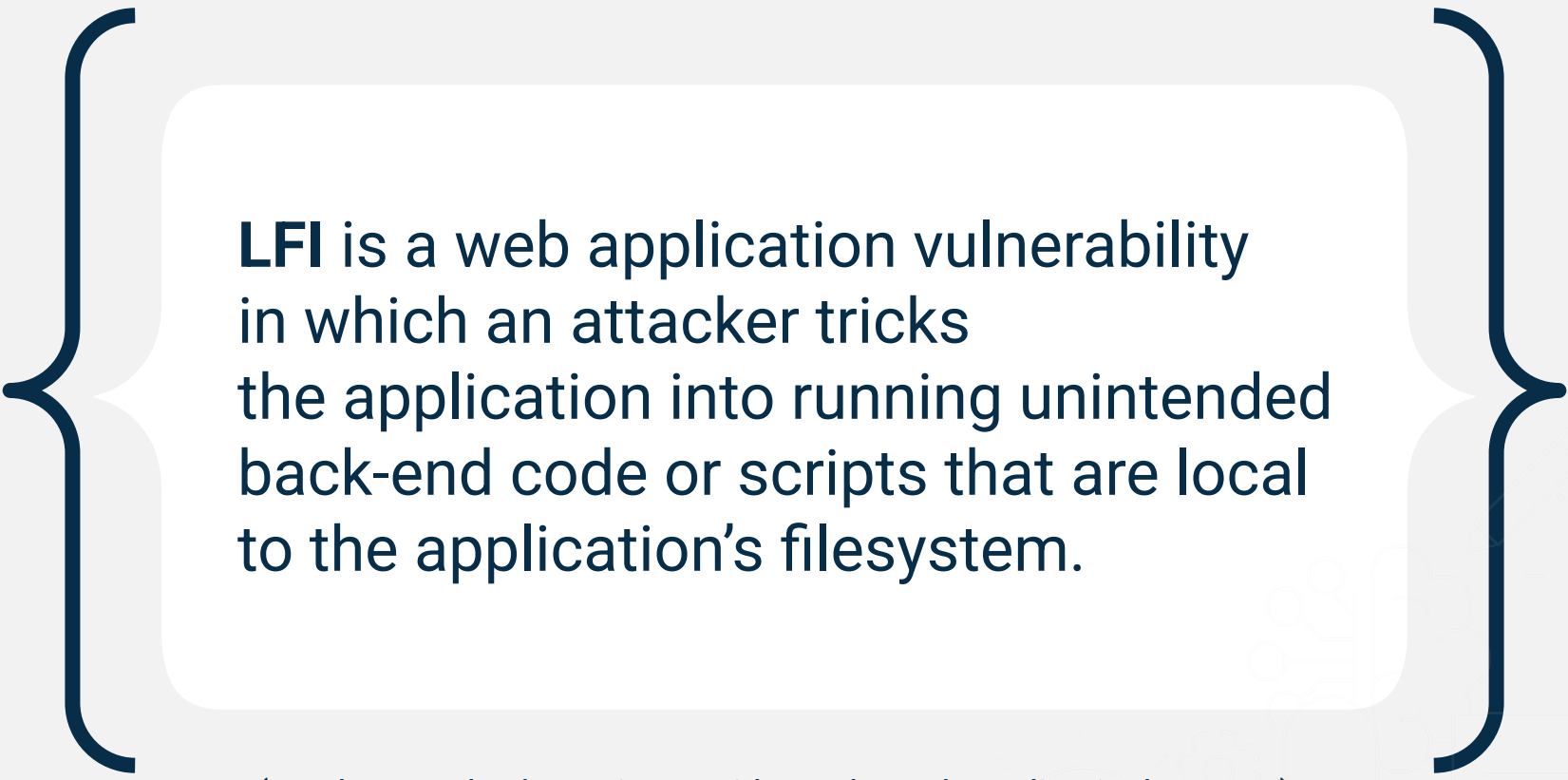
Directory traversal is a back-end component vulnerability in which an attacker accesses files and directories from a web application outside a user's authorized permissions.



NOTE: While directory traversal is a back-end component vulnerability that can cause significant impact, the direct impact is limited to an attacker viewing unauthorized data.



Another type of back-end component vulnerability is called **local file inclusion (LFI)**. LFI can let attackers view unauthorized data, impact the modification and deletion of data, and cause system outages.



LFI is a web application vulnerability in which an attacker tricks the application into running unintended back-end code or scripts that are local to the application's filesystem.

(In other words, the scripts reside on the web application's server.)

LFI

LFI is typically conducted by using a file upload feature to upload malicious scripts into the application's local filesystem. Common examples of file upload web applications include:

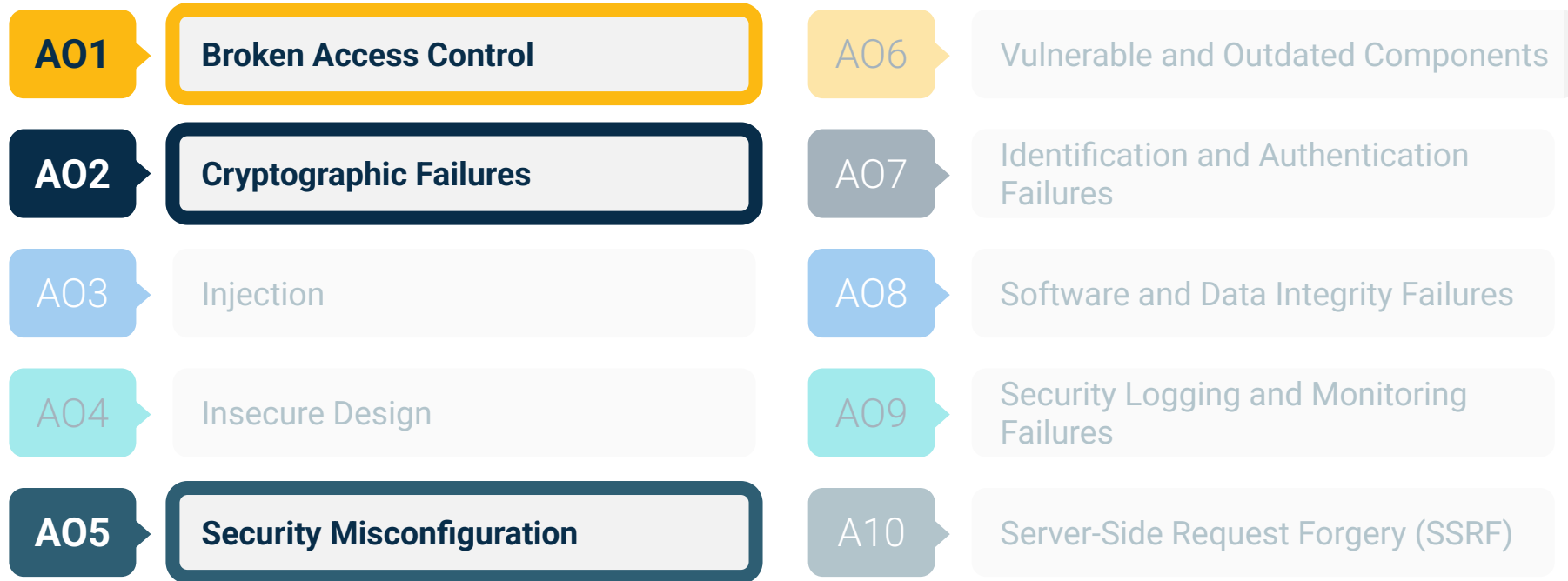
Uploading your resume for
an online job listing.



Uploading and viewing images
on a social media website.



Depending on how the attack is conducted, LFI could fall under the following three **OWASP Top 10** risks:





Before learning how to conduct an LFI exploit, we need to review how a web application runs back-end code like PHP.

PHP

PHP stands for **Hypertext Preprocessor**.

PHP is a server-side language used to develop web applications.

PHP files have a `.php` file extension.

PHP is often used to do the following:

- Connect to back-end databases such as MySQL
- Create and work with website cookies

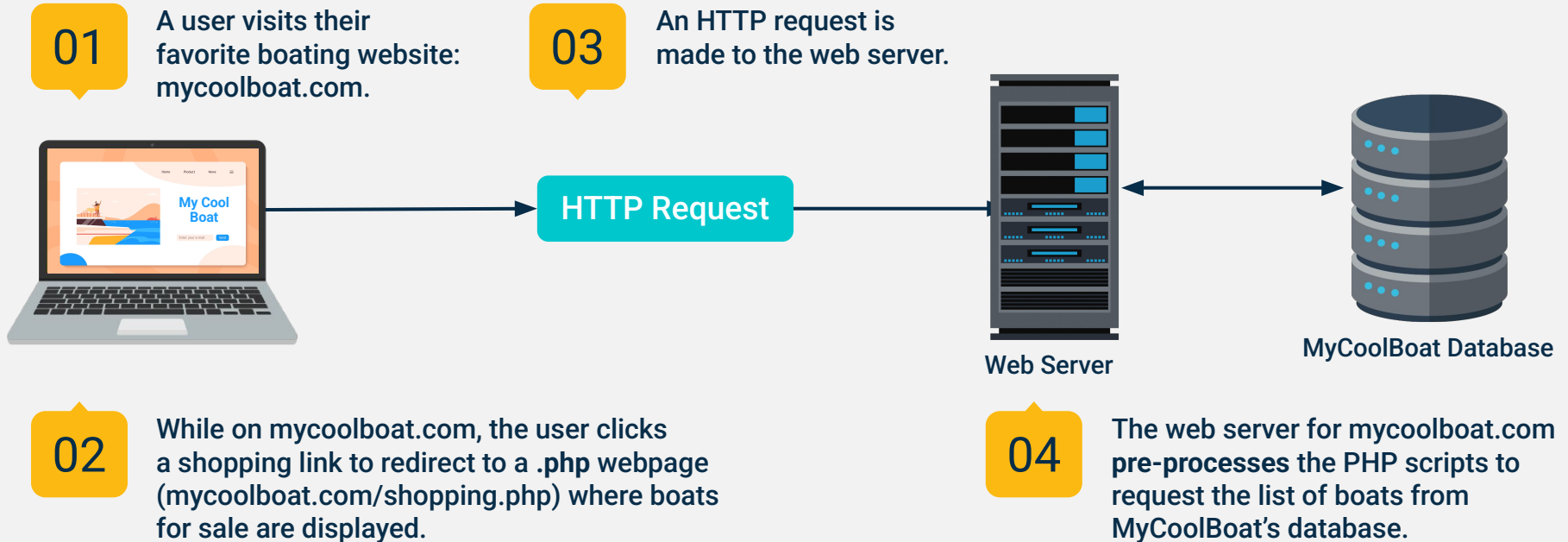
The PHP logo, featuring the lowercase letters 'php' in a bold, italicized, black font with a white outline, set against a dark blue oval background. The entire logo is positioned on a light blue diagonal band that runs from the top right towards the bottom left of the slide.

php



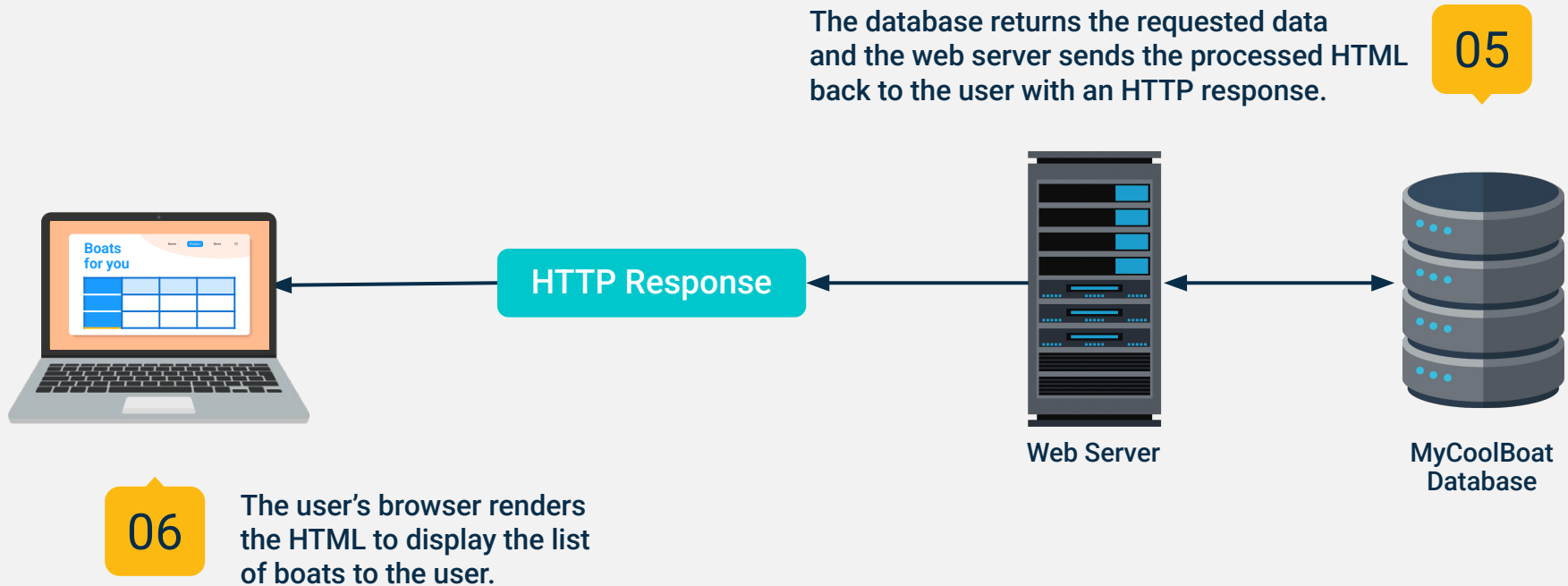
PHP Example

Suppose a user wants to display different types of boats at mycoolboat.com. The different types of boats are stored in a database on MyCoolBoat's servers.



PHP Example

Suppose a user wants to display different types of boats at mycoolboat.com. The different types of boats are stored in a database on MyCoolBoat's servers.



Takeaways

In the previous example:

- 1 The browser initiated a request to run the PHP script simply by accessing the page.
- 2 The PHP script exists and runs on the back-end server.



Now we know that PHP is a language that provides back-end functionality to a webpage, let's explore how LFI can use this functionality to cause unintended results.



Local File **Inclusion**

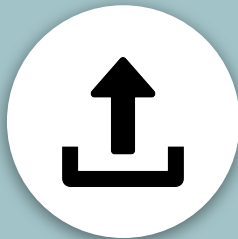
LFI Demonstration

In this demonstration, we will:



Illustrate

how a web application can use PHP and HTML together.



Show

the intended purpose of a web application's file upload functionality.



```
<script></script>
```

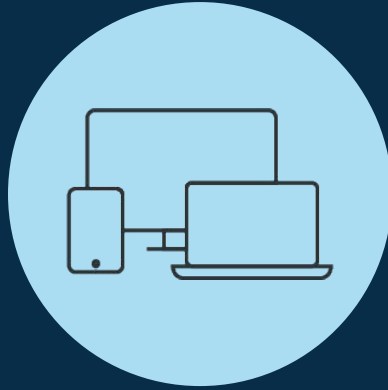
Upload

a malicious PHP script.



Run

command-line commands with the malicious PHP script.



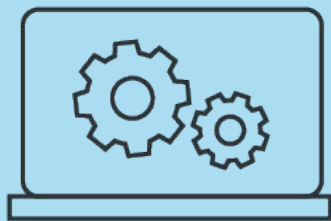
Instructor Demonstration

LFI

Demonstration Summary

The ability to successfully upload and run this script indicates that this web application is vulnerable to LFI.

- 1 We were able to include (upload) a file (script) locally, then run that unintended script.
- 2 This web application is also vulnerable to remote code execution (RCE).
- 3 RCE is a type of attack in which a malicious actor can execute arbitrary commands.



Real-World **Challenges,** **Impact,** and **Mitigations**

Real-World Challenges

Injection involves submitting an untrusted input. How applications work with the input depends on their design, architecture, and functionality. We'll focus on:

Our Demonstration

Illustrated using one type of malicious script and two command-line commands.

Thousands of malicious scripts can be uploaded and many malicious commands can be run.

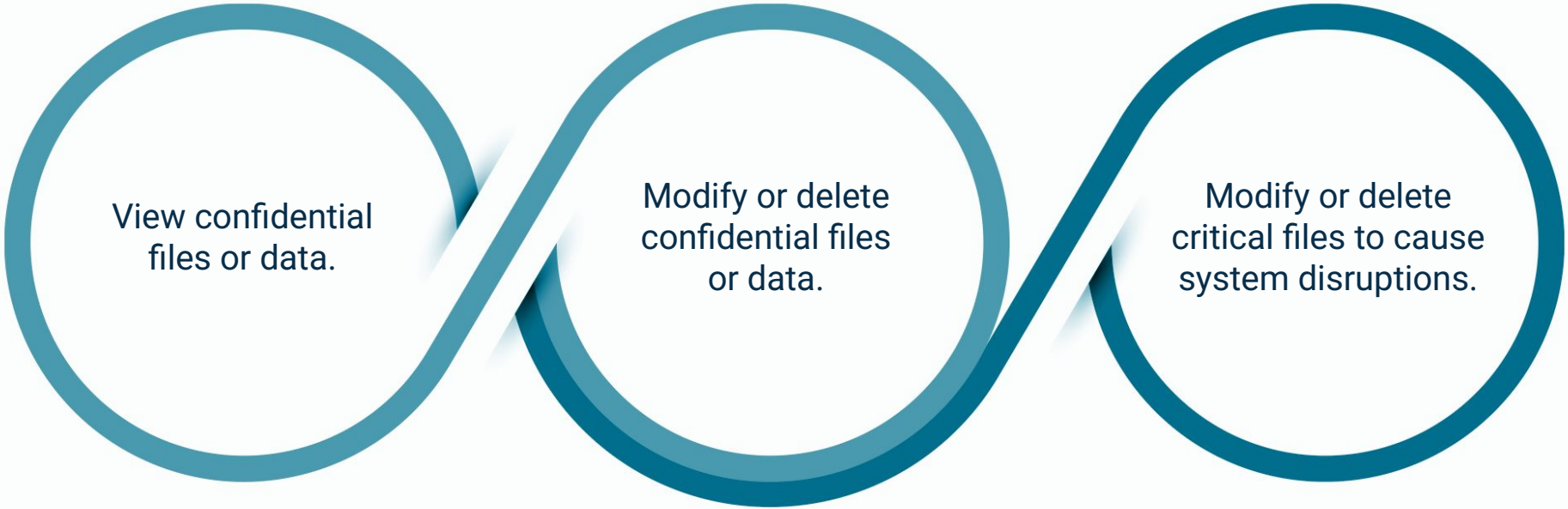
Our Attack Method

Used the PHP language, but this attack can also be accomplished with other languages, such as ASP.



Impact

If a malicious actor can upload a malicious script, they can potentially:



View confidential
files or data.

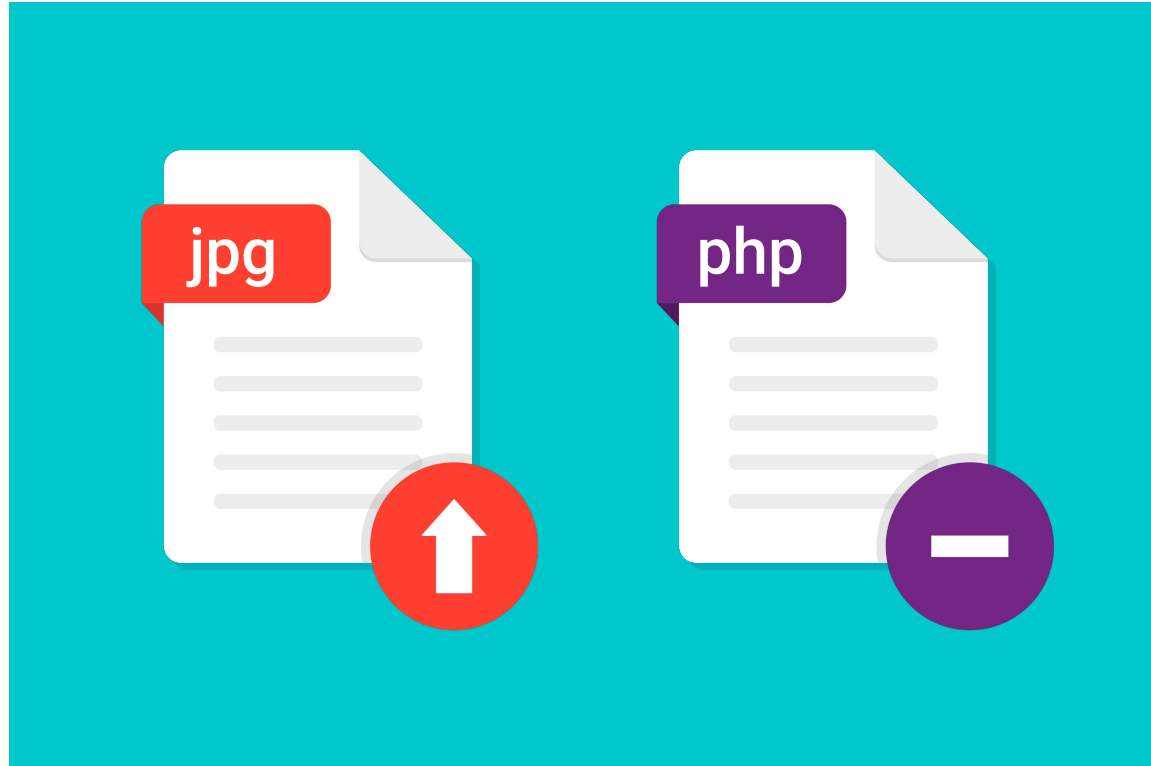
Modify or delete
confidential files
or data.

Modify or delete
critical files to cause
system disruptions.

Mitigation Methods

The best way to protect from LFI is to restrict users from being able to upload files into the local filesystem.

If the application requires file upload functionality, then the application should use allow listing to ensure that only a specific file type (like **.jpg**) can be uploaded, and not script file types such as **.php**.



LFI Summary

Intended purpose of original function	Web applications use file upload functionalities to enable a user to upload files into their local filesystem.
Vulnerability and Method of exploit	LFI is a vulnerability in which a malicious user can use the file upload functionality to upload unintended scripts into the web server's local filesystem.
Unintended consequence of the exploit	After the malicious script has been uploaded, a malicious user can run the script to cause unintended consequences.
Mitigation of the vulnerability	Limit file upload capabilities or use allow listing to restrict unintended file types.
Potential impact of the exploit	A malicious actor can access or modify confidential data or system files, or potentially cause system outages.



Questions?





Activity

LFI

You will continue in the role of an application security analyst working for bioengineering company Replicants.

In this activity, you'll inspect Replicants' main production website for LFI vulnerabilities.

Suggested Time
20 minutes



Time's Up!
Let's review



Questions?





Remote File Inclusion

Remote File Inclusion

So far we've covered:

Directory Traversal

A back-end component vulnerability in which an attacker accesses files and directories from a web application outside a user's authorized permissions.

LFI

Another back-end component vulnerability in which an attacker tricks the application into running unintended back-end code or scripts that are LOCAL to the application's filesystem.

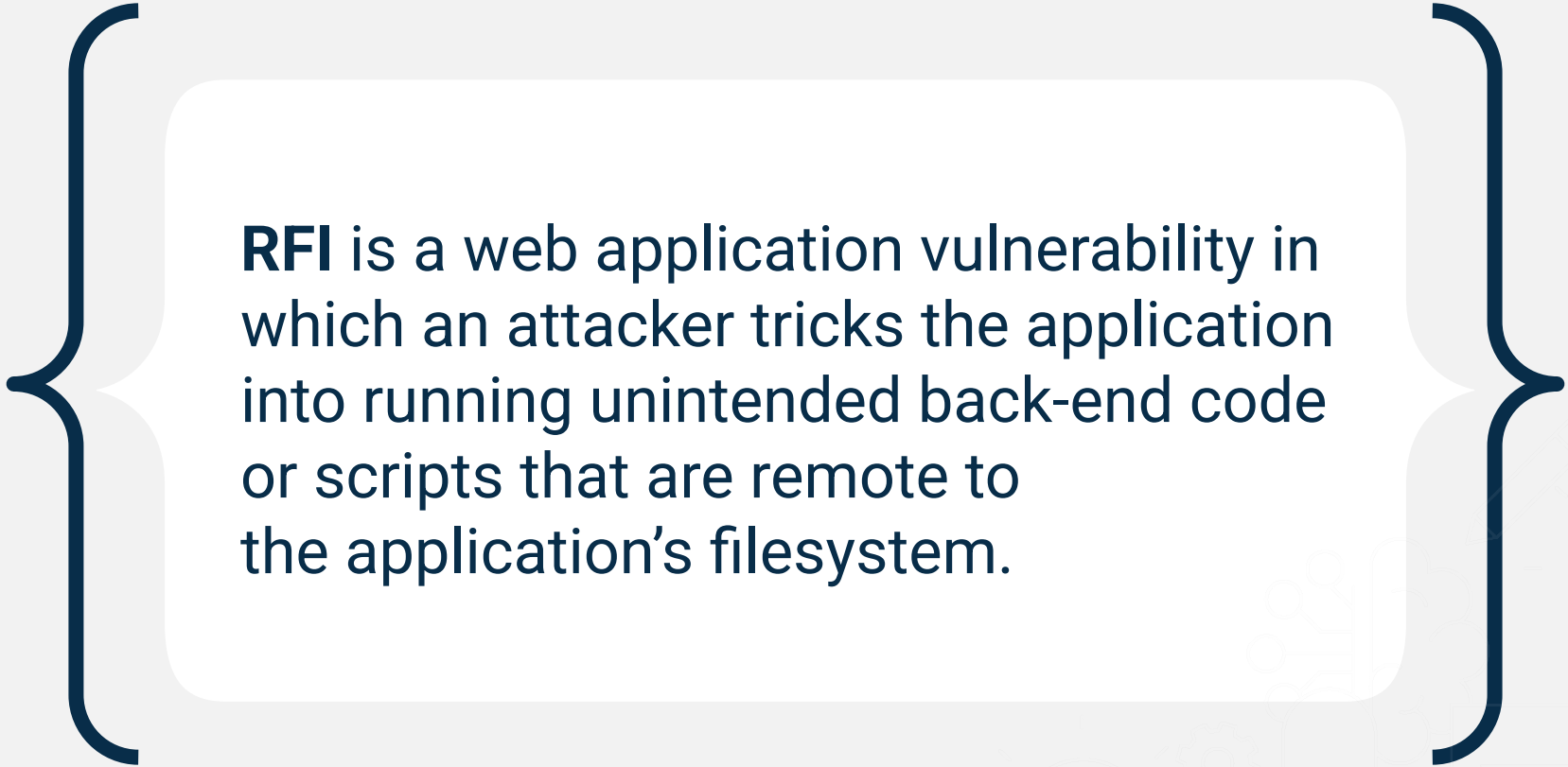
LFI depends on allowing a user to use unintended scripts that are LOCAL to the web server.

An application preventing malicious users from adding files to their local system **does not guarantee** that it is protected from an attacker running unintended back-end code or scripts.



Another type of back-end component vulnerability called **remote file inclusion (RFI)** involves using remote files or scripts to conduct a similar attack.

```
..._mod = modifier_ob.  
... mirror object to mirror_  
... mirror_mod.mirror_object =  
... operation == "MIRROR_X":  
... mirror_mod.use_x = True  
... mirror_mod.use_y = False  
... mirror_mod.use_z = False  
... operation == "MIRROR_Y":  
... mirror_mod.use_x = False  
... mirror_mod.use_y = True  
... mirror_mod.use_z = False  
... operation == "MIRROR_Z":  
... mirror_mod.use_x = False  
... mirror_mod.use_y = False  
... mirror_mod.use_z = True  
  
... selection at the end -add  
... _ob.select= 1  
... _ob.select=1  
... context.scene.objects.active  
... ("Selected" + str(modifier_  
... mirror_ob.select = 0  
... bpy.context.selected_object  
... data.objects[one.name].select  
  
... print("please select exactly  
  
... OPERATOR CLASSES -----  
  
... types.Operator):  
... X mirror to the selected  
... object.mirror_mirror_x"  
... mirror X"  
  
... context):  
... context.active_object is not
```



RFI is a web application vulnerability in which an attacker tricks the application into running unintended back-end code or scripts that are remote to the application's filesystem.

(In other words, the scripts DO NOT reside on the web application's server.)

A close-up photograph of a person's hands typing on a white laptop keyboard. In the background, a tablet displays a code editor with JavaScript code. The code includes comments and function calls related to game logic, such as 'player.velocity', 'player.position', and 'player.rotation'. A yellow circular graphic is overlaid on the right side of the image, containing text about Remote File Inclusion (RFI).

RFI is typically conducted
by modifying the parameter
of a URL to point to
a **remote script**.

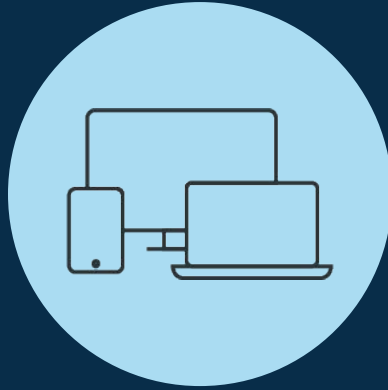
Depending on how the attack is conducted, RFI could fall under the following three **OWASP Top 10** risks:

A01	Broken Access Control	A06	Vulnerable and Outdated Components
A02	Cryptographic Failures	A07	Identification and Authentication Failures
A03	Injection	A08	Software and Data Integrity Failures
A04	Insecure Design	A09	Security Logging and Monitoring Failures
A05	Security Misconfiguration	A10	Server-Side Request Forgery (SSRF)

RFI Demonstration

In this demonstration, we will conduct the following three steps:

- 1 Revisit how a web application uses parameters as intended.
- 2 Modify the parameters to point to a remote site to cause unintended consequences.
- 3 Modify the URL to run this remote malicious PHP script.



Instructor Demonstration

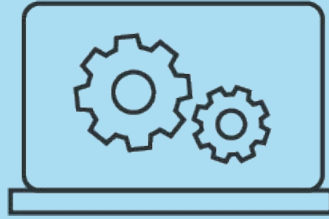
RFI

Demo Summary

The ability to successfully change the parameter of the URL to point to a remote script and then subsequently running this script indicates that this web application is vulnerable to RFI.

The ability to arbitrarily execute command-line code with the remote script means this web application is also vulnerable to RCE.

```
../../../../hackable/uploads/image.jpg successfully uploaded!
```



Real-World Challenges, Impact, and Mitigations

Real-World Challenges

Limitation of the RFI Attack

To conduct this attack successfully, the web server must run PHP and also needs to enable a configuration setting to allow external URLs.

Many web servers that run PHP do not have this configuration enabled.

Variety of Attacks

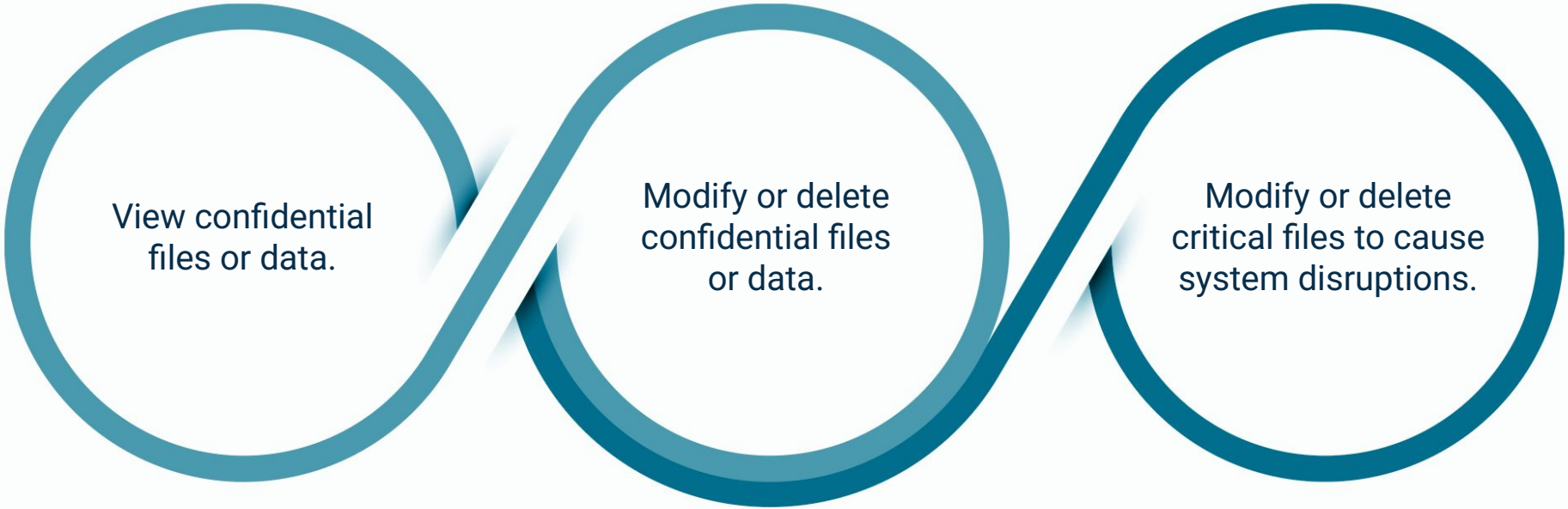
The demonstration illustrated using one type of malicious script and two command-line commands.

Thousands of malicious scripts can be uploaded and many malicious commands can be run.

The attack method used the PHP language, but this attack can also be accomplished with other languages, such as ASP.

Impact

If a malicious actor can remotely access a malicious script, they can:



View confidential
files or data.

Modify or delete
confidential files
or data.

Modify or delete
critical files to cause
system disruptions.

Mitigation Methods

The best way to protect from RFI is to avoid passing user-submitted input to your web server. This can be accomplished by using input validation.

Label

Invalid Input



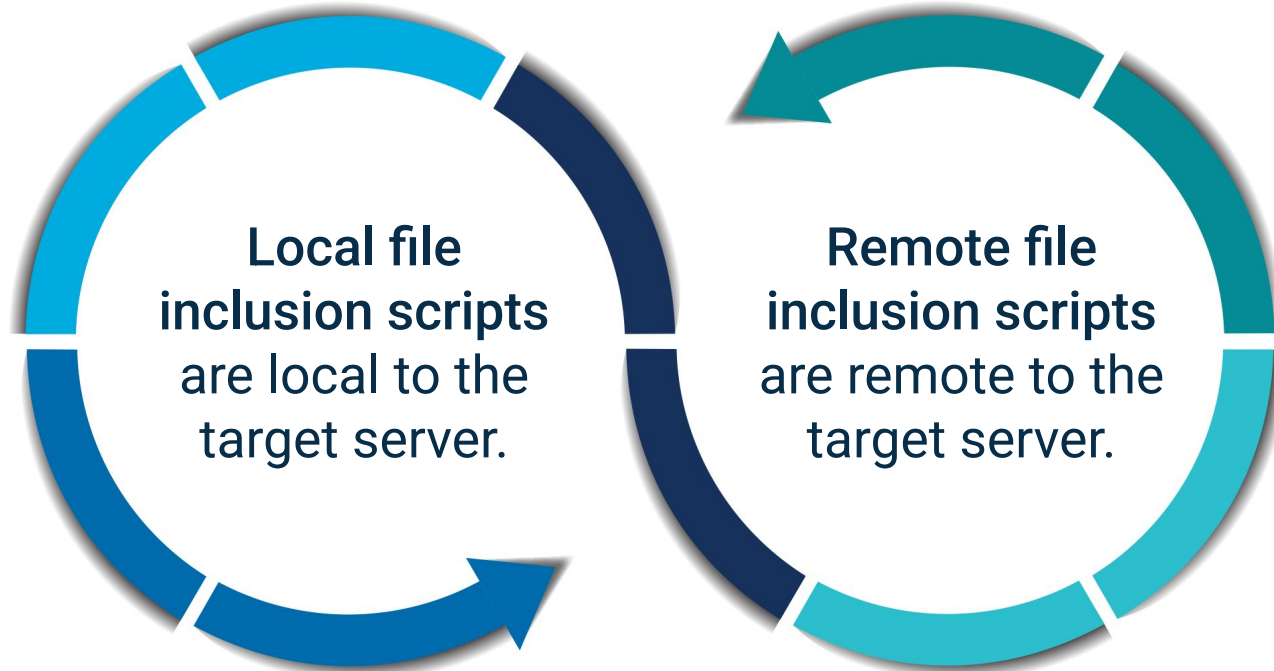
Error message with clear instruction

RFI Summary

Intended purpose of the original function	A web application can use parameters in its URL to indicate to the back-end server to access a certain resource.
Vulnerability and method of exploit	RFI is a vulnerability in which a malicious user can modify this parameter so that the back-end server accesses a malicious remote resource, such as a script.
Unintended consequence of the exploit	After the malicious remote script has been referenced, a malicious user can run the script to cause unintended consequences.
Mitigation of the vulnerability	Use input validation to avoid malicious submitted input, such as a modified parameter referencing a remote resource.
Potential impact of the exploit	A malicious actor can view or modify confidential data or system files, or potentially cause system outages.

Recap Back-End Components

Both of these attacks use back-end components of a web server to run unintended scripts, but they have some differences:





Questions?





Activity

RFI

In this activity, you will test Replicants' main production website for RFI vulnerabilities.

Suggested Time
20 minutes





Time's Up!
Let's review



Questions?





Thank You